

1. Query to return the customer_name, product_name, and total_amount for each sale in the last 30 days

```
SELECT
    c.customer_name,
    p.product_name,
    s.total_amount
FROM
    Sales s
JOIN
    Customers c ON s.customer_id = c.customer_id
JOIN
    Products p ON s.product_id = p.product_id
WHERE
    s.sale_date >= CURRENT_DATE - INTERVAL 30 DAY;
```

2. Query to find the total revenue generated by each product category in the last year

```
SELECT
    p.category,
    SUM(s.total_amount) AS total_revenue
FROM
    Sales s
JOIN
    Products p ON s.product_id = p.product_id
WHERE
    s.sale_date >= CURRENT_DATE - INTERVAL 1 YEAR
GROUP BY
    p.category;
```

3. Query to return all customers who made purchases in 2023 and are located in the "West" region

```
SELECT DISTINCT
    c.customer_name
FROM
    Sales s
JOIN
    Customers c ON s.customer_id = c.customer_id
WHERE
    YEAR(s.sale_date) = 2023 AND c.sales_region = 'West';
```

4. Query to display the total number of sales, total quantity sold, and total revenue for each customer

```
SELECT
    c.customer_name,
    COUNT(s.sales_id) AS total_sales,
    SUM(s.quantity) AS total_quantity,
    SUM(s.total_amount) AS total_revenue
FROM
    Sales s
```

```
JOIN
  Customers c ON s.customer_id = c.customer_id
GROUP BY
  c.customer_name;
```

5. Query to find the top 3 customers (by total revenue) in the year 2023

```
SELECT
  c.customer_name,
  SUM(s.total_amount) AS total_revenue
FROM
  Sales s
JOIN
  Customers c ON s.customer_id = c.customer_id
WHERE
  YEAR(s.sale_date) = 2023
GROUP BY
  c.customer_name
ORDER BY
  total_revenue DESC
LIMIT 3;
```

6. Query to rank products by their total sales quantity in 2023

```
SELECT
  p.product_name,
  SUM(s.quantity) AS total_quantity_sold,
  RANK() OVER (ORDER BY SUM(s.quantity) DESC) AS rank
FROM
  Sales s
JOIN
  Products p ON s.product_id = p.product_id
WHERE
  YEAR(s.sale_date) = 2023
GROUP BY
  p.product_name;
```

7. Query that categorizes customers into "New" or "Existing" based on their sign_up_date

```
SELECT
  c.customer_name,
  c.sales_region,
  CASE WHEN c.sign_up_date >= CURRENT_DATE - INTERVAL 6 MONTH THEN 'New'
        ELSE 'Existing'
  END AS customer_category
FROM
  Customers c;
```

8. Query to return the month and year along with the total sales for each month for the last 12 months

```
SELECT
```

```

    DATE_FORMAT(s.sale_date, '%Y-%m') AS month_year,
    SUM(s.total_amount) AS total_sales
FROM
    Sales s
WHERE
    s.sale_date >= CURRENT_DATE - INTERVAL 12 MONTH
GROUP BY
    DATE_FORMAT(s.sale_date, '%Y-%m')
ORDER BY
    DATE_FORMAT(s.sale_date, '%Y-%m');

```

9. Query to return product categories that generated more than \$50,000 in revenue during the last 6 months

```

SELECT
    p.category,
    SUM(s.total_amount) AS total_revenue
FROM
    Sales s
JOIN
    Products p ON s.product_id = p.product_id
WHERE
    s.sale_date >= CURRENT_DATE - INTERVAL 6 MONTH
GROUP BY
    p.category
HAVING
    SUM(s.total_amount) > 50000;

```

10. Query to check for any sales where the total_amount doesn't match the expected value (i.e., quantity * price)

```

SELECT
    s.sales_id,
    c.customer_name,
    p.product_name,
    s.quantity * p.price AS expected_total,
    s.total_amount AS actual_total,
    CASE WHEN (s.quantity * p.price != s.total_amount)
    THEN 'Mismatch' ELSE 'Correct' END

```